PROGRAMACIÓN III - ACTIVIDADES REFACTORIZADAS CON ESTRUCTURAS Y LISTAS

1. Actividad 1 - Ejercicio 1: Árbol Genealógico (Familia Simpson)

Modificaciones realizadas:

• Se utiliza la estructura persona(Nombre, Genero) en sustitución de los hechos hombre/1 y mujer/1.

• Se utiliza la estructura familia(Padre, Madre, [ListaHijos]) para agrupar núcleos familiares mediante listas.

• La determinación de parentescos y hermandades se realiza evaluando la pertenencia a las listas con member/2, eliminando duplicados.

Código en Prolog:

```
% Estructura persona(Nombre, Genero)
persona(abraham,    m).
persona(clancy,     m).
persona(herbert,    m).
persona(homero,     m).
persona(bart,       m).
persona(mona,       f).
persona(jacqueline, f).
persona(marge,      f).
persona(patty,      f).
persona(selma,      f).
persona(lisa,       f).
persona(maggie,     f).
persona(ling,       f).

% Estructura familia(Padre, Madre, [ListaHijos])
familia(abraham, mona,       [herbert, homero]).
familia(clancy,  jacqueline, [marge, patty, selma]).
familia(homero,  marge,      [bart, lisa, maggie]).
familia(ninguno, selma,      [ling]).

% Reglas de clasificación
hombre(X) :- persona(X, m).
mujer(X)  :- persona(X, f).

% Progenitores mediante listas
padre(P, H) :- familia(P, _, Hijos), P \= ninguno, member(H, Hijos).
madre(M, H) :- familia(_, M, Hijos), member(H, Hijos).

progenitor(P, H) :- padre(P, H).
progenitor(M, H) :- madre(M, H).

hijo(H, P) :- hombre(H), progenitor(P, H).
hija(H, P) :- mujer(H),  progenitor(P, H).

% Abuelos y nietos
abuelo(A, N) :- hombre(A), progenitor(A, Z), progenitor(Z, N).
abuela(A, N) :- mujer(A),  progenitor(A, Z), progenitor(Z, N).

nieto(N, A)  :- hombre(N), (abuelo(A, N) ; abuela(A, N)).
nieta(N, A)  :- mujer(N),  (abuelo(A, N) ; abuela(A, N)).

% Hermanos procesados sobre la lista familiar
hermanos(X, Y) :-
```

```
    familia(_, _, Hijos),
    member(X, Hijos),
    member(Y, Hijos),
    X \= Y.

hermano(X, Y) :- hombre(X), hermanos(X, Y).
hermana(X, Y) :- mujer(X),  hermanos(X, Y).

% Tíos y sobrinos
tio(T, S)     :- hombre(T), hermanos(T, P), progenitor(P, S).
tia(T, S)     :- mujer(T),  hermanos(T, P), progenitor(P, S).
sobrino(S, T) :- hombre(S), progenitor(P, S), hermanos(P, T).
sobrina(S, T) :- mujer(S),  progenitor(P, S), hermanos(P, T).

% Primos
primos(X, Y) :-
    progenitor(P1, X),
    progenitor(P2, Y),
    hermanos(P1, P2),
    X \= Y.

% Ancestros
ancestro(A, D) :- progenitor(A, D).
ancestro(A, D) :- progenitor(A, Z), ancestro(Z, D).
```

Consultas de validación:

?- abuelo(X, bart). % Retorna: X = abraham ; X = clancy.

?- hermana(X, bart). % Retorna: X = lisa ; X = maggie.

?- primos(bart, ling). % Retorna: true.

?- ancestro(X, bart). % Retorna todos los ancestros directos e indirectos.

2. Actividad 2 - Ejercicio 3: Grafo Dirigido de Ciudades de Canadá

Modificaciones realizadas:

• Se encapsulan los arcos en la estructura arista(Origen, Destino, Costo).

• Se incorporan listas tanto en la obtención de adyacencias como en el rastreo de caminos visitados y reconstrucción de rutas.

• Se preservan las reglas de aristas salientes, costos intermedios y búsqueda dirigida.

Código en Prolog:

```
% Base de conocimiento con estructura arista(Origen, Destino, Costo)
arista(vancouver, edmonton, 16).
arista(vancouver, calgary, 13).
arista(edmonton,  saskatoon, 12).
arista(calgary,   edmonton, 4).
arista(calgary,   regina, 14).
arista(saskatoon, calgary, 9).
arista(saskatoon, winnipeg, 20).
arista(regina,    saskatoon, 7).
arista(regina,    winnipeg, 4).

% Compatibilidad con consultas arco/3
arco(Origen, Destino, Costo) :- arista(Origen, Destino, Costo).

% Regla C: Existencia y lista de aristas salientes
tiene_aristas(Nodo) :- arista(Nodo, _, _).

aristas_salientes(Nodo, ListaDestinos) :-
    findall(destino(Dest, Costo), arista(Nodo, Dest, Costo), ListaDestinos),
    ListaDestinos \= [].

% Regla D: Costo vía nodo intermedio
costo_via_intermedio(X, Y, Z, CostoTotal) :-
    arista(X, Y, C1),
    arista(Y, Z, C2),
    CostoTotal is C1 + C2.

% Recorrido dirigido gestionado con listas de visitados y ruta
camino_dirigido(Origen, Destino, Ruta, Costo) :-
    construir_camino(Origen, Destino, [Origen], RInvertida, Costo),
    reverse(RInvertida, Ruta).

construir_camino(Origen, Destino, Visitados, [Destino|Visitados], Costo) :-
    arista(Origen, Destino, Costo),
    \+ member(Destino, Visitados).

construir_camino(Origen, Destino, Visitados, RutaFinal, CostoTotal) :-
    arista(Origen, Siguiente, C1),
    \+ member(Siguiente, Visitados),
    construir_camino(Siguiente, Destino, [Siguiente|Visitados], RutaFinal, C2),
    CostoTotal is C1 + C2.
```

Consultas de validación:

?- camino_dirigido(saskatoon, vancouver, Ruta, Costo). % Retorna: false.

?- arco(regina, Destino, Costo). % Retorna destinos y costos desde Regina.

?- tiene_aristas(vancouver). % Retorna: true.

?- costo_via_intermedio(vancouver, edmonton, saskatoon, C). % Retorna: C = 28.

?- camino_dirigido(edmonton, calgary, Ruta, Costo). % Retorna: Ruta = [edmonton, saskatoon, calgary], Costo = 21.